2026年　IoTスクール

TA向け追加資料

SYSTEMCREATORS

本日の内容

当日子供たちが行う資料に沿って作業し、予習を行ってください。

資料URL：https://colab.research.google.com/github/YukiAoki-GU/IoT_for_beginners/blob/main/2026_IoT_school.ipynb

▶ 短縮URL **https://x.gd/Qxy43**

補 足

システムクリエイターズの Node-RED はTA用アカウントを使用してください。

アクセス先

https://ai-server.msep.jp/t26○○ （t2601～t2605）

ログイン情報

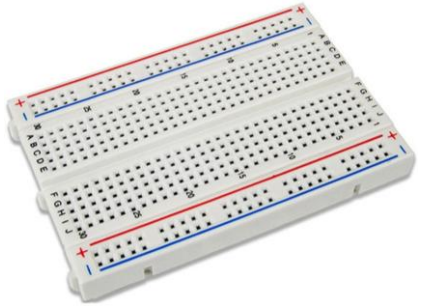
ユーザー名： **t26○○** / パスワード： **password**

通常はRaspberry Piの裏側のユーザー名に沿って
ai-server.msep.jp/[ユーザー名] にアクセスします。
今回はTA予習用に専用アカウントを用意しました。

内容物一覧

Raspberry PI

ブレッドボード



USBハブ×1

HDMIアダプター

OTGケーブル

マイクロUSB
アダプター

マイクロUSB
スイッチケーブル

電源アダプタ

LED

ジャンパー線 4本

その他

※一部の箱にのみ入っています

ドライバ

ピンヘッダ

HDMIケーブル

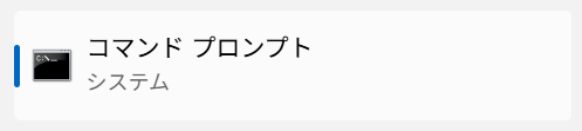
Raspberry Pi の起動時にNode-REDにアクセスできない場合

① 起動に失敗している可能性があるので、LEDの点滅が落ち着いたら一度電源を切って再起動してみる

② PCがラズパイと同じネットワーク（上毛新聞WiFi：dchalle）につながっていることを確認

③ ラズパイが Wifi につながっているかを確認

- PC のコマンドプロンプトを開いて ping コマンドを送ります。
  ping s26○○.local とコマンドを入力します。



→応答がない場合、ラズパイがWiFiにつながっていない可能性があるのでネットワークに繋ぎます。
④を行います。

```
C:\Users\SC005>ping s2626.local
ping 要求ではホスト s2626.local が見つかりませんでした。ホスト名を確認してもう一度実行してください。
```

→応答があった場合、Node-RED が正しく起動していない可能性があるので、再起動を行います。
⑤を行います。

```
C:\Users\SC005>ping s2625.local

s2625.local [2400:2412:4d6d:1410:fa30:277a:e5ac:73b9]に ping を送信しています 32 バイトのデータ:
2400:2412:4d6d:1410:fa30:277a:e5ac:73b9 からの応答: 時間 =4ms
2400:2412:4d6d:1410:fa30:277a:e5ac:73b9 からの応答: 時間 =7ms
2400:2412:4d6d:1410:fa30:277a:e5ac:73b9 からの応答: 時間 =9ms
2400:2412:4d6d:1410:fa30:277a:e5ac:73b9 からの応答: 時間 =19ms

2400:2412:4d6d:1410:fa30:277a:e5ac:73b9 の ping 統計:
    パケット数: 送信 = 4、受信 = 4、損失 = 0 (0% の損失)、
ラウンド トリップの概算時間 (ミリ秒):
    最小 = 4ms、最大 = 19ms、平均 = 9ms
```

Raspberry Pi の起動時にNode-REDにアクセスできない場合

④ ラズパイに繋ぐための小型のモニタが会場にあるはずなので、それをつないでラズパイの画面を表示します。
画面右上の WiFi 設定から会場のWiFi に繋いでください。

Raspberry Pi の起動時にNode-REDにアクセスできない場合

⑤ 違う方法でラズパイにアクセスし、状態を確かめます。

- PC のコマンドプロンプトを開いて SSH コマンドでラズパイにアクセスします。
  ssh s26○○@s26○○.local とコマンドを入力します。 ここでアクセスできない場合は、④ を行ってください。

```
C:\Users\SC005>ssh s2625@s2625.local
The authenticity of host 's2625.local (2400:2412:4d6d:1410:fa30:277a:e5ac:73b9)' can't be established.
ED25519 key fingerprint is SHA256:mhPoN3wBoz8q03Sloz8MLHK5bS4spzUPJ/MGfouM2Tk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
Warning: Permanently added 's2625.local' (ED25519) to the list of known hosts.
s2625@s2625.local's password:
```

初めての接続時には、接続するかどうかを確認されます。「**yes**」と入力します。

パスワードは「**password**」です。
入力しても何も表示されないので注意

```
s2625@s2625:~ $
```

ログインに成功するとこのような表示になります。

アクセスしたら、Node-RED の起動コマンドを入力します。
sudo systemctl enable –now nodered.service コマンドを入力した後、
Node-REDにアクセスできるかを確認してください。

```
s2625@s2625:~ $ sudo systemctl enable --now nodered.service
```

その他の考えられる問題

MQTTサーバーにつながらない

症状 MQTTがいつまでも接続中でつながらない

対処

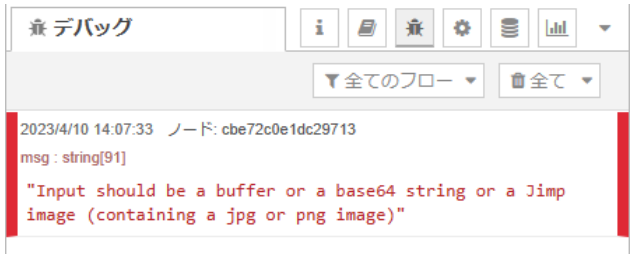
- サーバー設定、ユーザー名、パスワードが正しいかを確認する

- 以下の設定でも接続ができるので試してみる

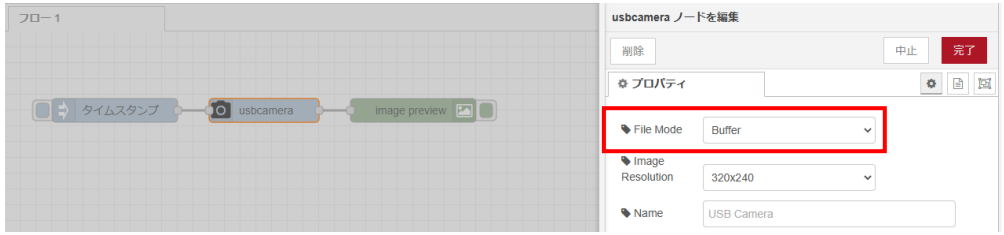
サーバー	ポート
ai-server.msep.jp/mqtt	443
ai-server.msep.jp/mqtt	1883
ai-server.msep.jp	1883

Node-REDからカメラで撮影ができない

症状 カメラで撮影しようとするとエラーが表示される



対処 File Mode を一度 File Mode に設定し、一度実行してから再度、Bufferに直して動作させる

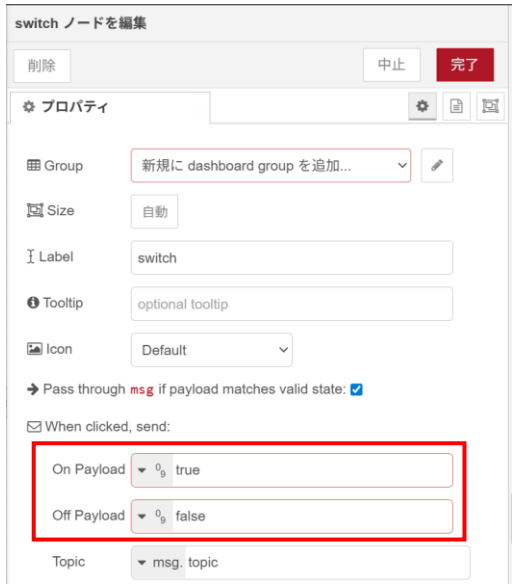


その他の考えられる問題

Node-RED の switch ノードで設定が変わらない

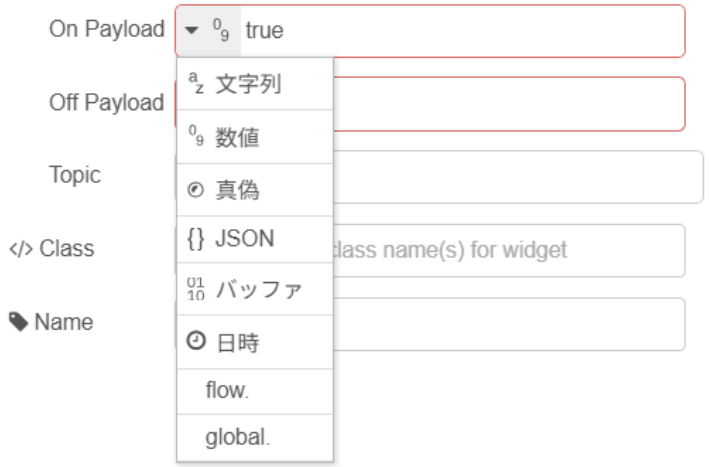
症状

switch ノード On Payload、Off Payload の値を変更しても元に戻ってしまう。



対処

一度、数値以外の設定に変更して完了、デプロイしてから再度、設定を行う。



追加で予習してほしい資料

カメラを使う高校生が多いため、以下の資料についても予習をお願いします。

・Teachable_Machine について

https://colab.research.google.com/github/cmd-gunma-univ/DX-highschool/blob/main/Teachable_Machineについて.ipynb

・YOLO使い方

https://colab.research.google.com/github/cmd-gunma-univ/DX-highschool/blob/main/YOLO使い方.ipynb

・YOLO練習1

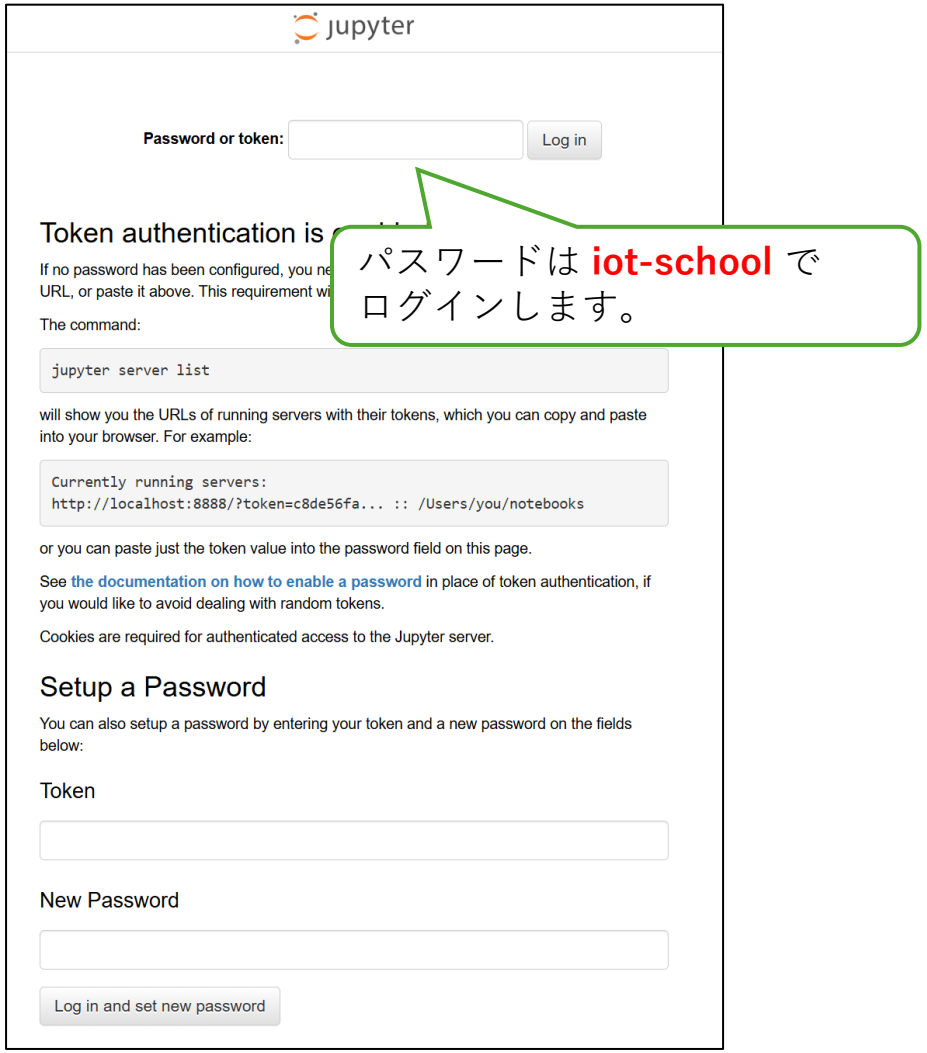
https://colab.research.google.com/github/cmd-gunma-univ/DX-highschool/blob/main/YOLO練習1(bbox).ipynb

・YOLO練習2

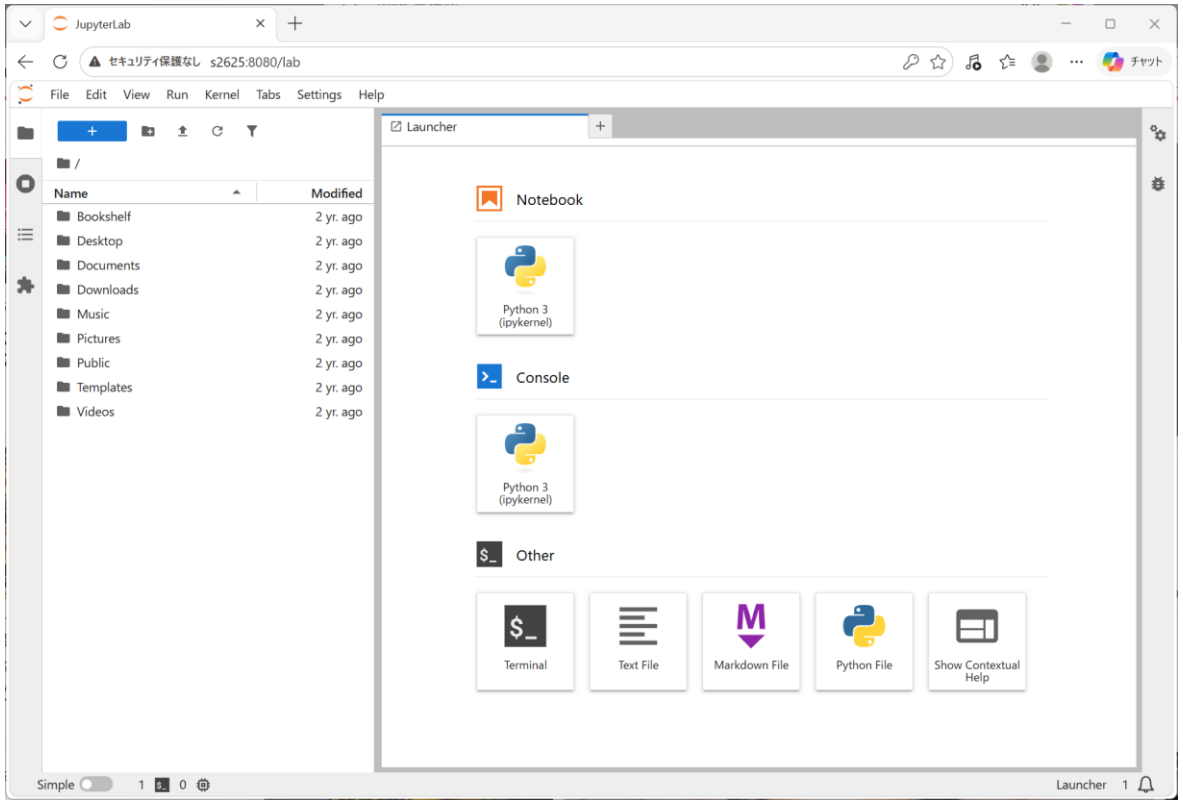
https://colab.research.google.com/github/cmd-gunma-univ/DX-highschool/blob/main/YOLO練習2(pose).ipynb

追加資料を行うために①

自由制作等でPythonを使う場合は、Node-REDと同様に Raspberry Pi 上で起動している Jupyter Lab にアクセスします。
URL：**http://[ユーザー名]:8080**



ログイン後

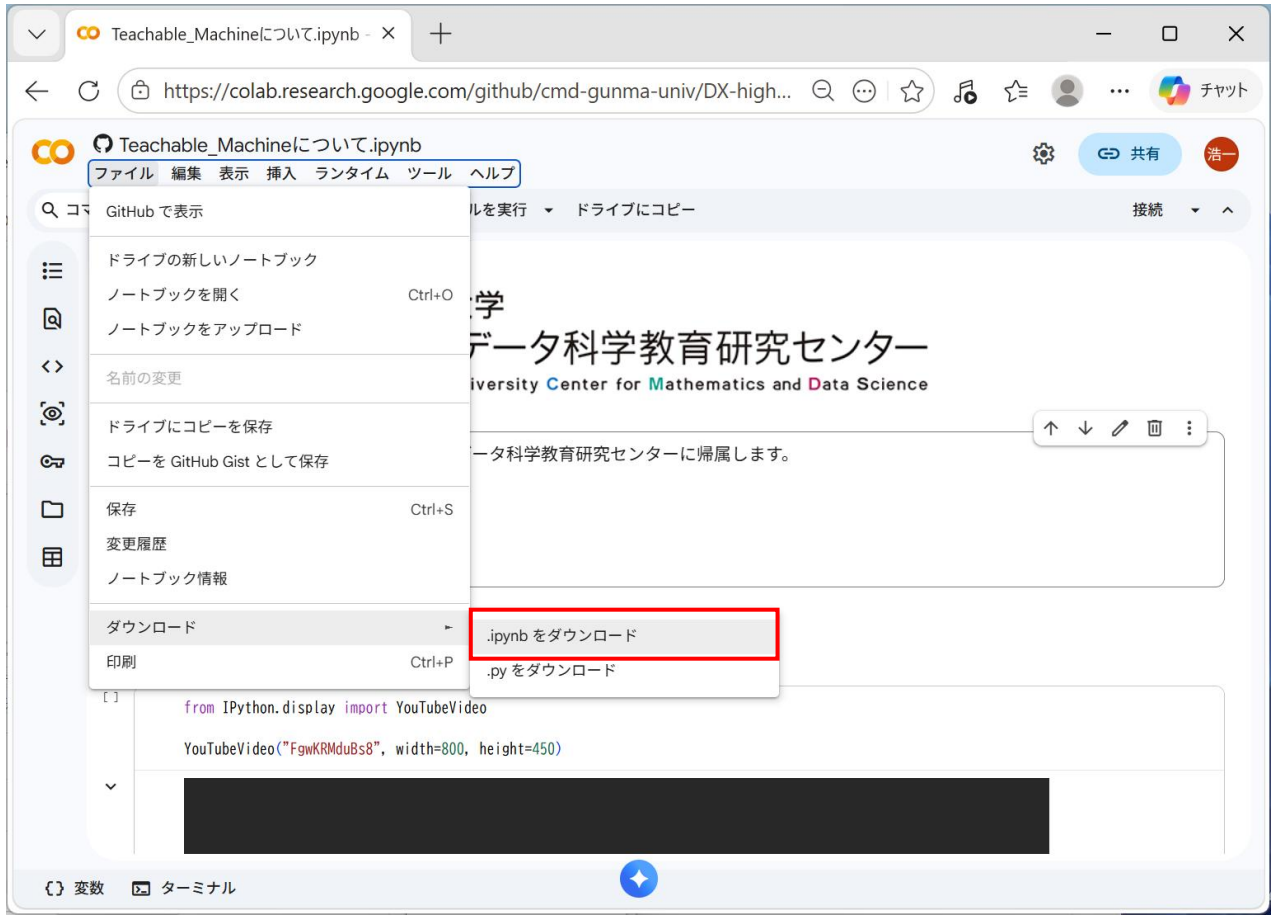


ⓘ **JupyterLabとは**
ブラウザ上でラズパイにアクセスし、Pythonを書いて実行できる開発環境。
ラズパイ内のファイルを見たり編集することもできる。

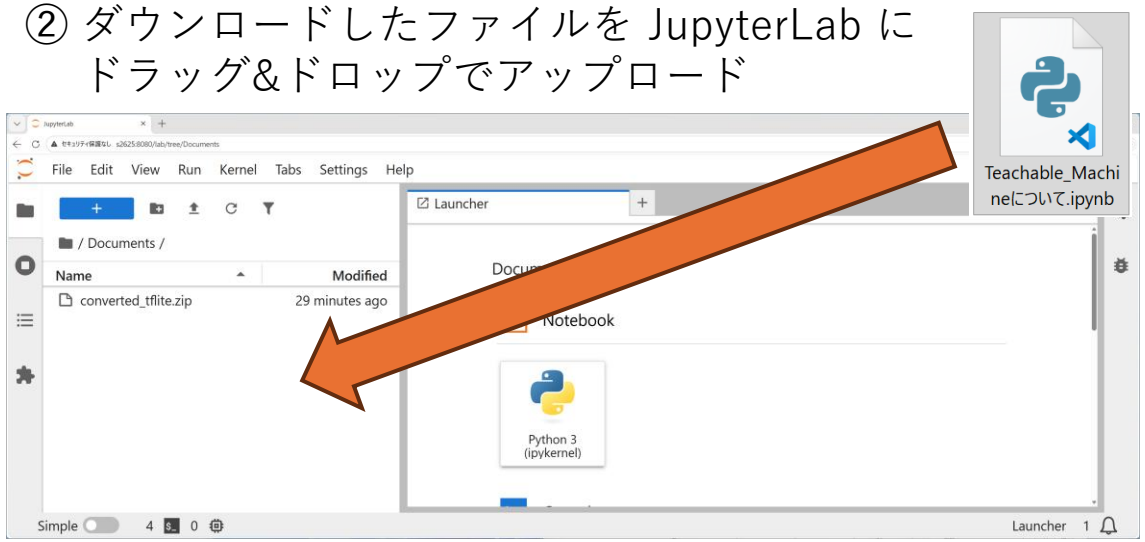
追加資料を行うために②

資料内では、ラズパイ上で資料を動かしています。資料と同じ操作ができるように資料をラズパイに入れてみましょう。

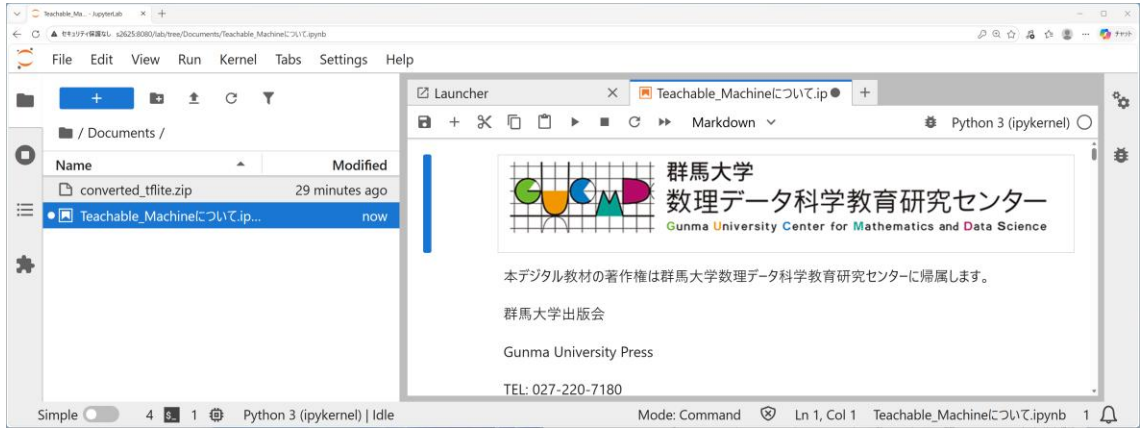
① 資料をブラウザで開き、
[ファイル]→[ダウンロード]→[.ipynbをダウンロード]を選びます。



② ダウンロードしたファイルを JupyterLab に
ドラッグ&ドロップでアップロード



③ ダブルクリックでファイルを開く

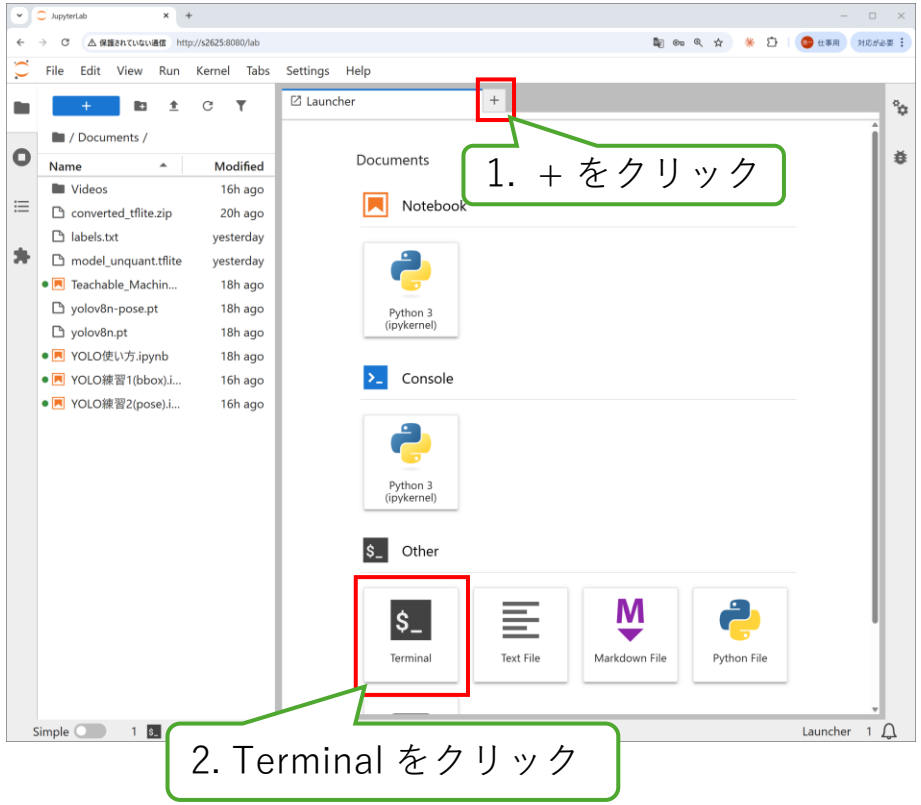


追加資料を行うために③

資料内でプログラムを動かすことがありますが、そのまま動かすと右のようなエラーが発生します。事前に必要なライブラリをインストールしましょう。

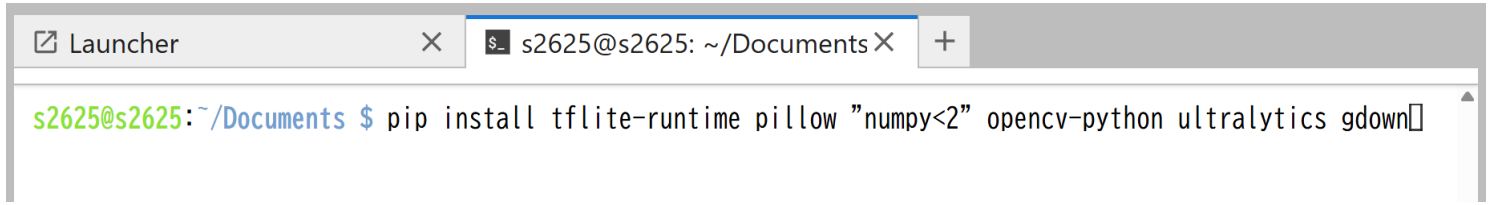
```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[2], line 4
      1 # Teachable Machine (TFLite) + USBカメラ (OpenCV) リアルタイム推論
      2 # pip install tflite-runtime pillow numpy opencv-python
----> 4 import cv2
      5 import numpy as np
      6 from PIL import Image

ModuleNotFoundError: No module named 'cv2'
```

① ターミナルを起動

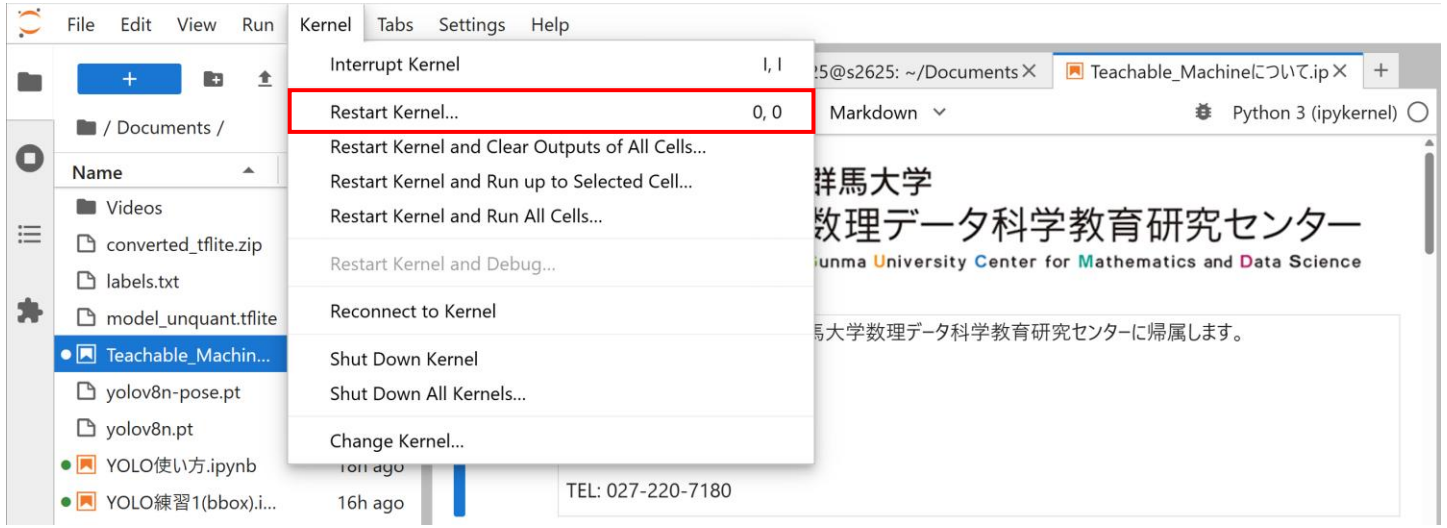


② ターミナルに以下のコマンドを入力

pip install tflite-runtime pillow “numpy<2” opencv-python ultralytics gdown



③ 資料を開き、[Kernel]→[Restert Kernel] をクリック



PythonとNode-REDを組み合わせる

Node-RED と Pythonプログラムを連携させられるようにしてみましょう。

① ipynb で試して動かす
〇〇.ipynb
セルごとに確認しながら開発

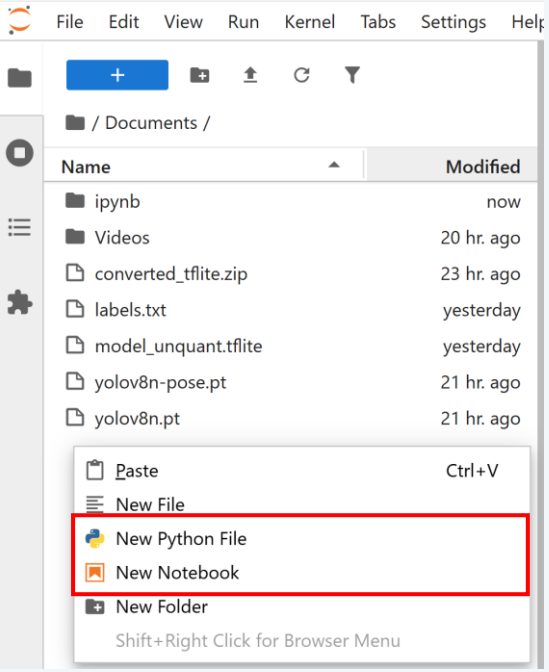
→

② .py に持ってくる
〇〇.py
動いたコードをpyファイルにコピー

→

③ Node-RED から起動
execノード
Node-REDからpyファイルを起動

1. ファイルブラウザの右クリックメニューから [**New Notebook**] で **ipynb ファイル**を作成
2. セルにPythonプログラムを作成
3. ファイルブラウザの右クリックメニューから [**New Python File**] で **pyファイル** を作成
4. ipynb のセルで **作成したコード** を **pyファイル** にコピペし、**Ctrl + S** で保存
5. Node-RED から呼び出す



PythonとNode-REDを組み合わせる（Node-REDからPythonファイルを実行する）

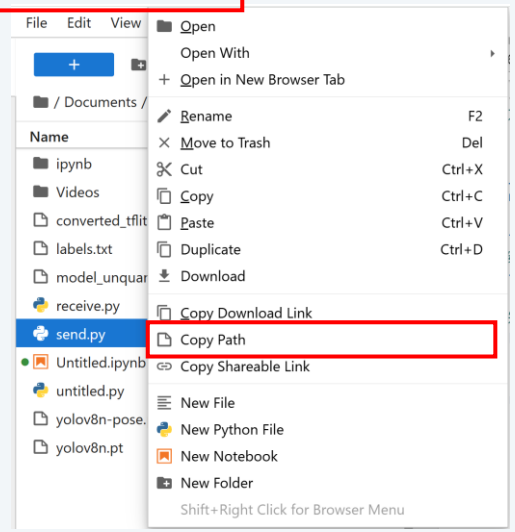
Node-RED から Python を起動する場合、 exec ノードを使用する

※ **exec** ノードは「OS のコマンドを実行する」ノードです。
Python スクリプトの起動コマンドを実行します。

- **コマンド：python3**
- **追加引数：/home/[ユーザー名]/[○○.pyのファイルパス]**
- **出力：コマンド実行中 – spawn モード**
- 出力ピンは3つ：
  - 上　＝**標準出力**（**print()** 結果）
  - 真ん中＝**標準エラー**（エラーメッセージ）
  - 下　＝**終了コード**

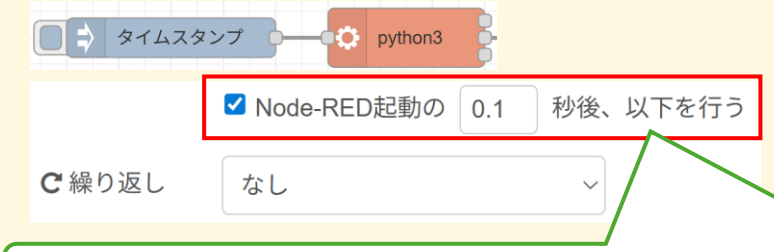
py ファイルのパスはJupyterLab で ファイルを右クリックし、 [Copy Path]でパスをコピーできます。

それに
/home/[ユーザー名]/
を頭につけて追加引数に設定します。



補足：Pythonスクリプトが自動で起動するように設定する

inject ノードを設定してラズパイが起動したら自動で pythonプログラムが起動するようにできます。



チェックを入れると自動で inject ノードが実行されます。

PythonとNode-REDを組み合わせる（MQTTでNode-REDとデータをやり取りする）

① ライブラリのインストール（最初の1回だけ・ターミナルで実行）

```
pip install paho-mqtt
```

② 接続先 MQTT ブローカー（WebSocket）

- ホスト：**ai-server.msep.jp/mqtt/** ／ ポート：**80**
- ユーザー名：**iot** ／ パスワード：**iot-school**

```python
#送る (publish)
import paho.mqtt.client as mqtt
# クライアント作成 (WebSocket)
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,
              transport="websockets")
c.ws_set_options(path="/mqtt/")
c.username_pw_set("iot", "iot-school")
# 接続
c.connect("ai-server.msep.jp", 80)
# 通信を安定させる (入れとけばOK)
c.loop_start()

# 送信 (完了まで待つ)
c.publish("s2625", "メッセージ").wait_for_publish()

# 切断
c.disconnect()
```

```python
#受け取る (subscribe)
import paho.mqtt.client as mqtt
# 受信時に呼ばれる関数
def on_message(c, u, m):
    print(m.topic, m.payload.decode())

c = mqtt.Client(transport="websockets")
c.ws_set_options(path="/mqtt/")
c.username_pw_set("iot", "iot-school")
# 受信処理を登録
c.on_message = on_message

# 接続
c.connect("ai-server.msep.jp", 80)
# 通信を安定させる (入れとけばOK)
c.loop_start()

# このトピックを受信
c.subscribe("s2625")
```

追加資料のソースコード簡単説明
（YOLO使い方.ipynb：USBカメラのチェック）

```python
# USBカメラでリアルタイム撮影
import cv2
from IPython.display import display, Image, clear_output
import time

# 0 はカメラID。複数接続時は 1, 2 に変更して試してください
cap = cv2.VideoCapture(0)

# 必要なら解像度指定（任意）
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)

try:
  if not cap.isOpened():
    raise RuntimeError("カメラが開けませんでした。/dev/video0 を確認してください。")

  while True:
    ret, frame = cap.read()
    if not ret:
      print("フレームを取得できませんでした。")
      break

    # Jupyterで表示(cv2.imshowは使わない)
    ok, jpg = cv2.imencode('.jpg', frame)
    if not ok:
      continue
    clear_output(wait=True)
    display(Image(data=jpg.tobytes()))
    time.sleep(0.03)  # 約30fps相当の待ち(調整可)

except KeyboardInterrupt:
  print("■ 手動で停止しました。")

finally:
  cap.release()
  print("📷 カメラを解放しました。")
```

カメラを開く。エラーが発生した場合は1や2を試しましょう。

カメラ解像度を設定します。カメラが対応していない場合は、近い解像度が選ばれます。

撮影された画像を1枚取り出します。
ret ：画像があるかどうか(True/False)、frame：画像

画像データを jpg データに変換。
ok ：成功かどうか(True/False)、jpg：JPG画像

clear_output：ipynb ファイルで表示している画像をリセット
display：ipynb ファイルで画像を表示

追加資料のソースコード簡単説明（YOLO使い方.ipynb：物体検出について）

```
# ===== YOLOモデル =====
model = YOLO("yolov8n.pt")  # 軽量・高速

 . . .

# ===== 高速化設定 =====
INFER_EVERY = 3       # 推論を間引く
DISPLAY_EVERY = 3     # 表示も間引く
frame_i = 0
last_annotated = None
try:
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_i += 1
        # ===== 推論 =====
        if frame_i % INFER_EVERY == 0:
            results = model.predict(
                source=frame,
                imgsz=320,
                conf=0.35,
                verbose=False
            )[0]
            # 全物体を描画
            last_annotated = results.plot()
        # ===== 表示 =====
        if frame_i % DISPLAY_EVERY == 0 and last_annotated is not None:
            ok, jpg = cv2.imencode(".jpg", last_annotated)
            if ok:
                clear_output(wait=True)
                display(Image(data=jpg.tobytes()))
```

YOLOv8の最軽量モデル(yolov8n)を読み込みます。

INFER_EVERYフレームに1回画像をAI処理します。

model.predict()でYOLOでAI処理。
- source：画像データを渡します。
- imgsz：渡した画像データを指定のサイズにリサイズして処理します。
  32の倍数にしてください。
- conf：検出の最低スコア。0.35 なら35%以上の確率で検出した
  オブジェクトだけが表示されます。
  間違った検出が多い/検出されない場合は調整しましょう。
- verbose：Trueだと検出ログが表示されます。

results.plot()で検出結果の枠とラベルを書いた画像を取得します。

DISPLAY_EVERY フレームに1回画像を表示します。

追加資料のソースコード簡単説明（YOLO使い方.ipynb：ポーズについて）

```
# ===== モデル =====
model = YOLO("yolov8n-pose.pt")

 . . .

# ===== 高速化パラメータ =====
INFER_EVERY = 3
DISPLAY_EVERY = 3
frame_i = 0
last_annotated = None
try:
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_i += 1
        # ===== 推論を間引く =====
        if frame_i % INFER_EVERY == 0:
            results = model.predict(
                source=frame,
                imgsz=256,   # 小さくする
                conf=0.35,
                verbose=False
            )[0]
            last_annotated = results.plot()
        # ===== 表示 =====
        if frame_i % DISPLAY_EVERY == 0 and last_annotated is not None:
            ok, jpg = cv2.imencode(".jpg", last_annotated,
                                   [int(cv2.IMWRITE_JPEG_QUALITY), 70])
            if ok:
                clear_output(wait=True)
                display(Image(data=jpg.tobytes()))
```

YOLOv8のポート検出用モデル(yolov8n-pose)を読み込みます。

INFER_EVERYフレームに1回画像をAI処理します。

model.predict()でYOLOでAI処理。
- source：画像データを渡します。
- imgsz：渡した画像データを指定のサイズにリサイズして処理します。
  32の倍数にしてください。今回の場合、
  320×240 → 256×192 にリサイズした画像が処理されます。
- conf：検出の最低スコア。0.35 なら35%以上の確率で検出した
  オブジェクトだけが表示されます。
  間違った検出が多い/検出されない場合は調整しましょう。
- verbose：Trueだと検出ログが表示されます。

results.plot()で「人の枠＋関節17点＋骨格ライン」を描いた画像を取得します。

DISPLAY_EVERY フレームに1回画像を表示します。

画像データを jpg データに変換。
画像品質を70に軽量化（デフォルト95）